



ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

D
BACH KHOA

TRÍ TUỆ NHÂN TẠO

N
A
N
G



Khoa Công Nghệ Thông Tin

TS. Nguyễn Văn Hiệu

TRÍ TUỆ NHÂN TẠO

Chương 2: Không gian trạng thái và tìm kiếm mù

D
BACH KHOA

N
A
N
G

Nội dung

- Giải quyết vấn đề
- Không gian trạng thái
- Tìm kiếm trên không gian trạng thái
- Tìm kiếm theo chiều rộng
- Tìm kiếm đều giá
- Tìm kiếm theo chiều sâu
- Tìm kiếm chiều sâu có giới hạn
- Bài tập

Giải quyết vấn đề

- Phát biểu chính xác bài toán
 - Hiện trạng ban đầu,
 - Kết quả mong muốn,..
- Phân tích bài toán.
- Thu thập và biểu diễn dữ liệu, tri thức cần thiết để giải bài toán.
- Lựa chọn kỹ thuật giải quyết thích hợp

Không gian trạng thái

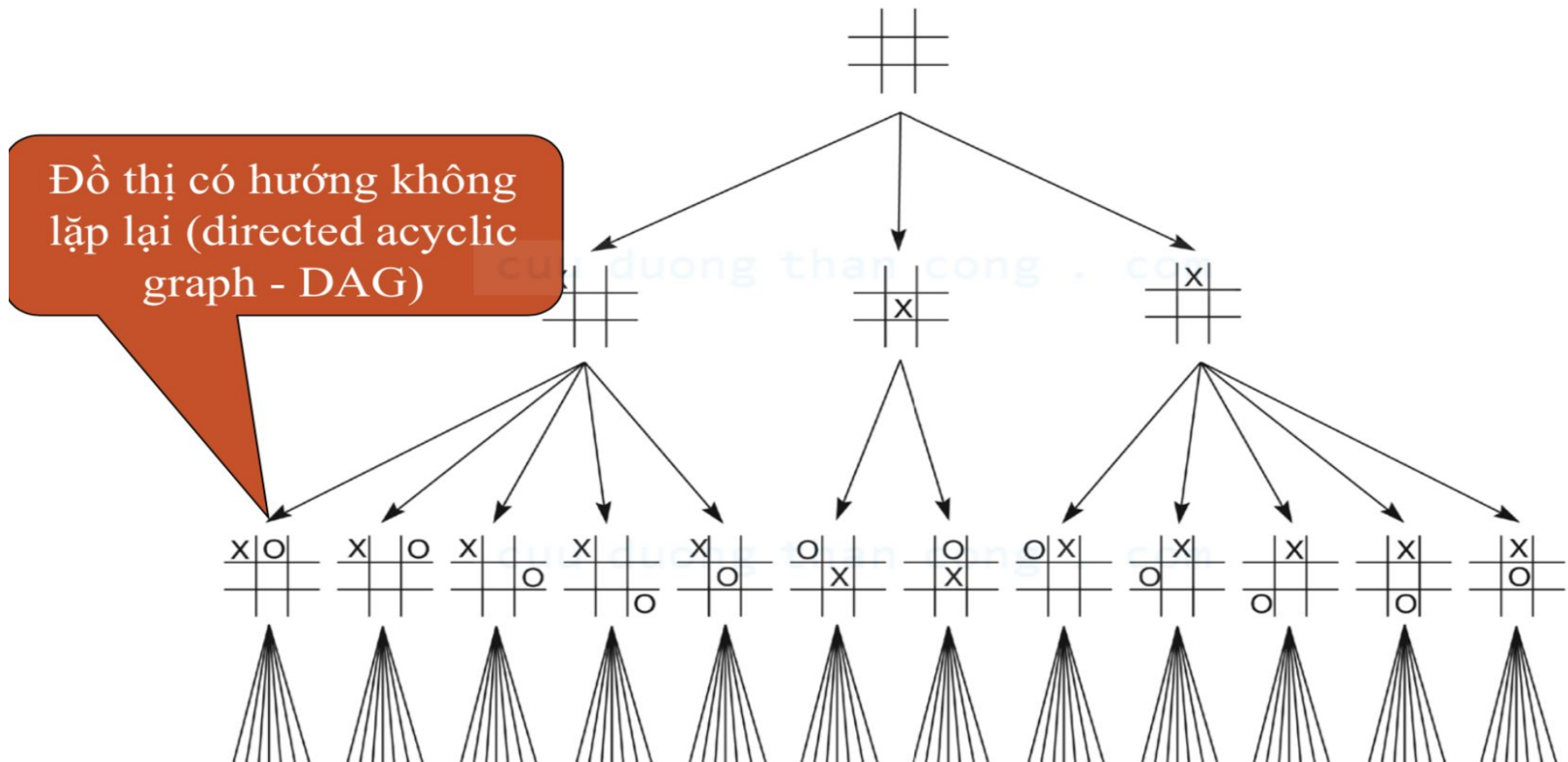
- Có nhiều cách để biểu diễn vấn đề
- Sử dụng đồ thị để biểu diễn vấn đề gọi là đồ thị không gian trạng thái
- Sử dụng lý thuyết đồ thị để phân tích cấu trúc và độ phức tạp của vấn đề
- Ví dụ: hệ thống cầu thành phố Königsberg và biểu diễn đồ thị tương ứng (Leonhard Euler)

Không gian trạng thái

- Một không gian trạng thái(state space) là 1 bộ $[N, A, S, G]$ trong đó:
 - N (node) là các đỉnh(các trạng thái) của đồ thị.
 - A (arc) là tập các cạnh hay cung giữa các đỉnh.
 - S (start) là tập trạng thái đầu.
 - G (Goal) chứa các trạng thái đích của bài toán ($S \rightarrow N \rightarrow S$).
- Các trạng thái trong G được mô tả theo một trong hai đặc tính:
 - Đặc tính có thể đo lường được các trạng thái gặp trong quá trình tìm kiếm. Ví dụ: Tic-tac- toe, 8-puzzle,...
 - Đặc tính của đường đi được hình thành trong quá trình tìm kiếm. Ví dụ : TSP
- Đường đi của lời giải (solution path) là một con đường đi từ một đỉnh thuộc S đến một đỉnh thuộc G .

Không gian trạng thái

- Một phần của không gian trạng thái trong trò chơi Tic-tac-toe



Không gian trạng thái

- Trò chơi 8 ô và trò chơi 15 ô
- Cách biểu diễn trạng thái của trò chơi này như thế nào ?

trạng thái đầu

11	14	4	7
10	6		5
1	2	13	15
9	12	8	3

trạng thái đích

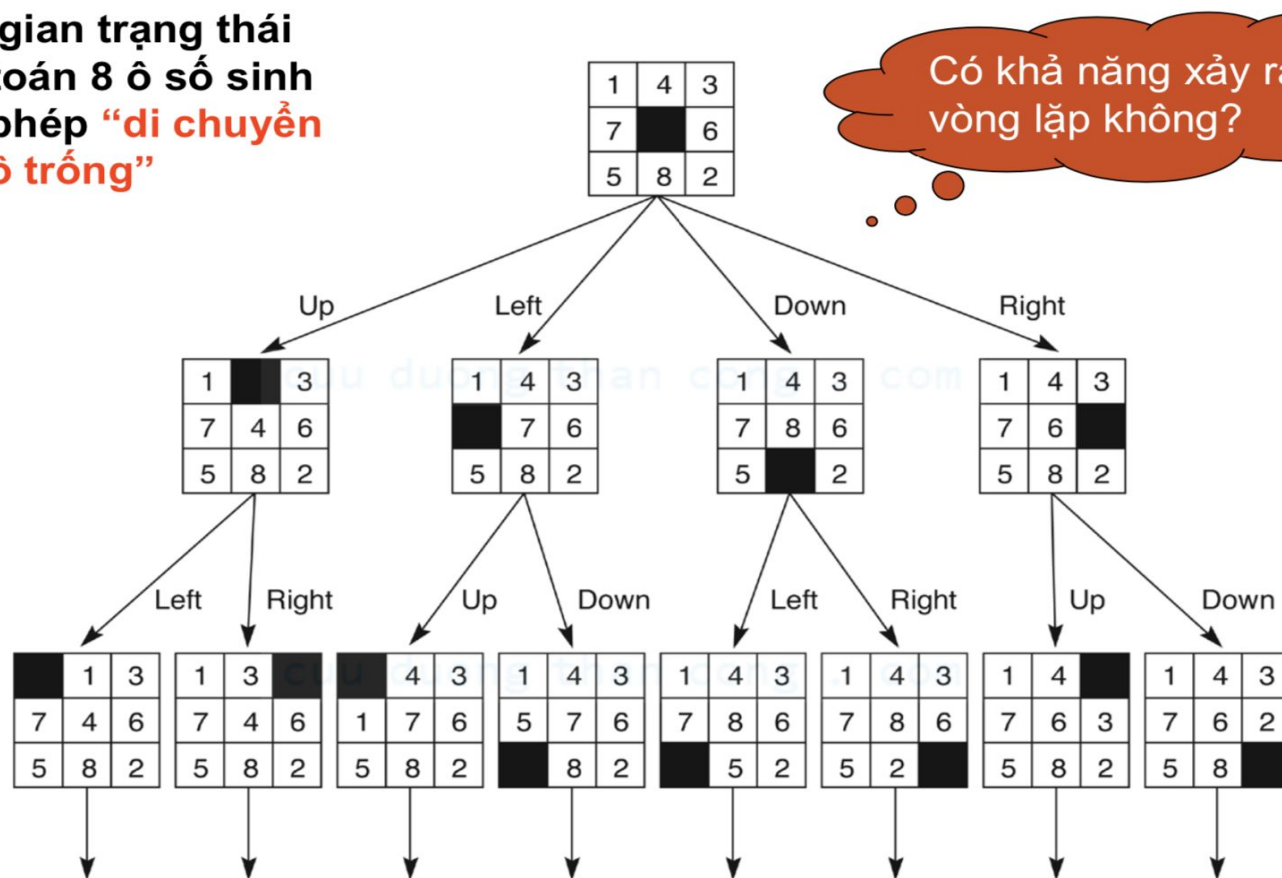
1	2	3	4
12	13	14	5
11		15	6
10	9	8	7

	2	8
3	5	7
6	2	1

1	2	3
8		4
7	6	5

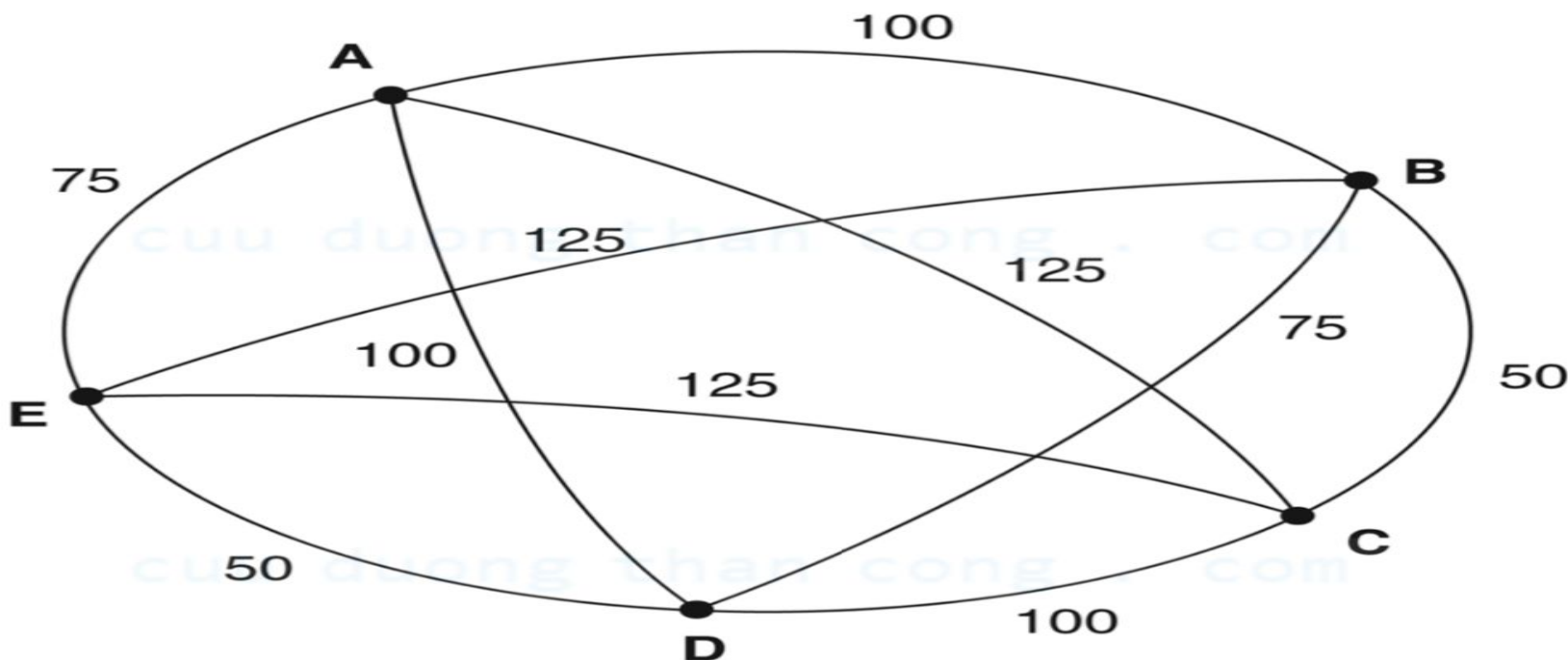
Không gian trạng thái

- Không gian trạng thái của bài toán 8 ô số sinh ra bằng phép “**di chuyển ô trống**”

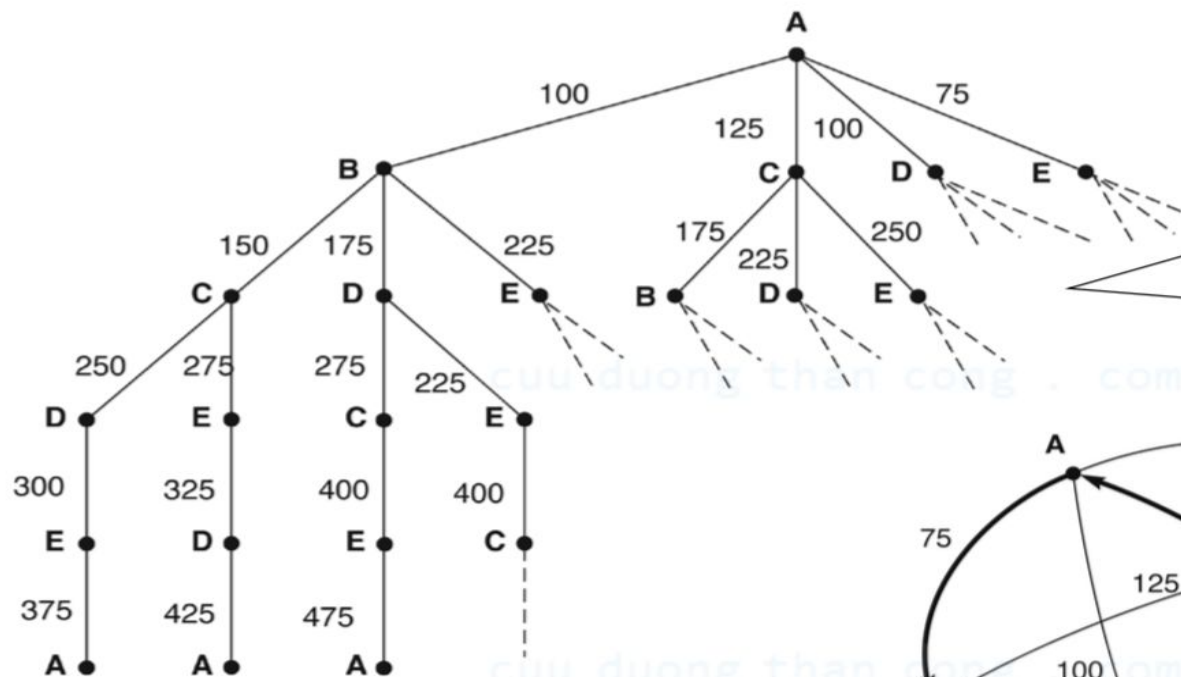


Không gian trạng thái

- Cần biểu diễn không gian trạng thái cho bài toán người đưa thư như thế nào?

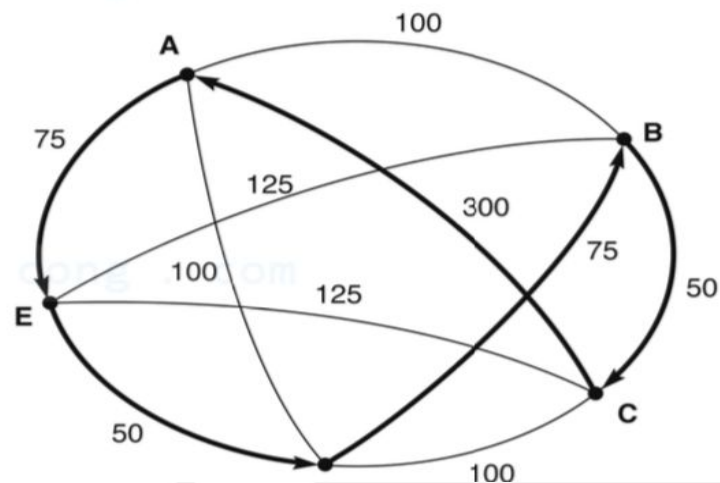


Không gian trạng thái



Mỗi cung được đánh dấu bằng tổng giá của con đường từ nút bắt đầu đến nút hiện tại.

Path: ABCDEA	Path: ABCEDA	Path: ABDCEA	...
Cost: 375	Cost: 425	Cost: 475	



Không gian trạng thái

- Bốn yếu tố chính để xác định không gian trạng thái
 - Trạng thái
 - Hành động
 - Kiểm tra trạng thái thỏa đích
 - Chi phí cho mỗi bước chuyển trạng thái

Tìm kiếm trong không gian trạng thái

- Chiến lược tìm kiếm là chiến lược lựa chọn thứ tự xét các đỉnh tạo ra.
- Các tiêu chuẩn để đánh giá chiến lược :
 - **đủ**: liệu có tìm được lời giải (nếu có)
 - **độ phức tạp thời gian**: số lượng đỉnh phải xét
 - **độ phức tạp lưu trữ**: tổng dung lượng bộ nhớ phải lưu trữ (các đỉnh trong quá trình tìm kiếm).
 - **tối ưu**: có luôn cho lời giải tối ưu.
- Độ phức tạp thời gian và lưu trữ của bài toán có thể được đo bằng:
 - **b**: độ phân nhánh của cây
 - **d**: độ sâu của lời giải ngắn nhất
 - **m**: độ sâu tối đa của không gian trạng thái (có thể vô hạn).

Tìm kiếm trong không gian trạng thái

- **Chiến lược tìm kiếm mù**

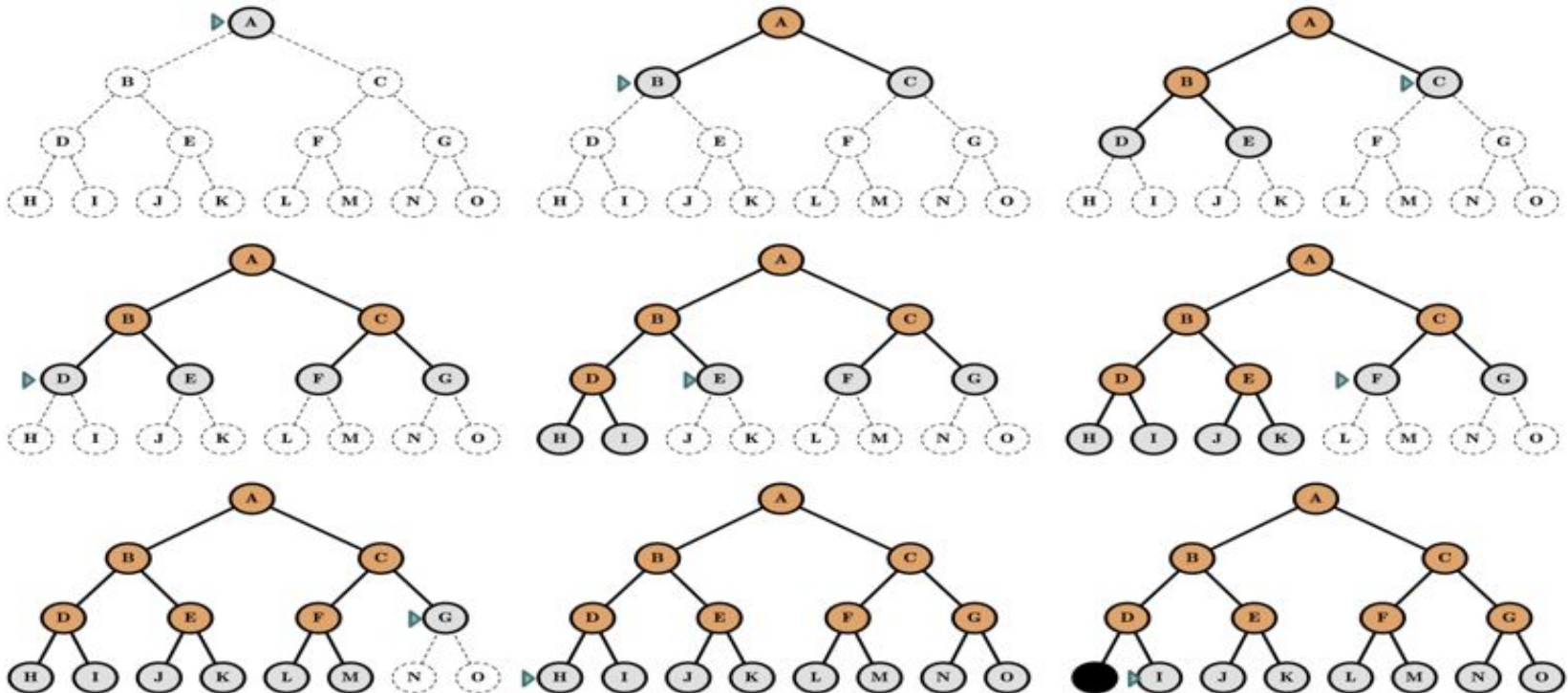
- Các chiến thuật tìm kiếm chỉ sử dụng thông tin từ định nghĩa bài toán
 - **Tìm kiếm theo chiều rộng**
 - Tìm kiếm đều giá (uniform-cost search)
 - **Tìm kiếm theo chiều sâu.**
 - Tìm kiếm theo chiều sâu có giới hạn

Tìm kiếm theo chiều rộng

- Tìm kiếm theo từng tầng. Phát triển các đỉnh gần với đỉnh hiện tại
- Tập đỉnh được chia thành 3 khu vực
 - Khu vực đã phát triển(explored): tập đỉnh đã xét
 - Khu vực chờ phát triển(frontier): tập đỉnh đang chờ xét
 - Khu vực chưa phát triển(unexplored): tập đỉnh chưa xét

Tìm kiếm theo chiều rộng

- Tìm kiếm theo từng tầng. Phát triển các đỉnh gần với đỉnh hiện tại
- Minh họa sự phát triển các đỉnh theo thuật toán tìm kiếm theo chiều rộng



Tìm kiếm theo chiều rộng

- Tìm kiếm theo từng tầng. Phát triển các đỉnh gần với đỉnh hiện tại
- Thuật toán tìm kiếm theo chiều rộng được mô tả bởi thủ tục:

```
function BREADTH-FIRST-SEARCH(initialState, goalTest)
    returns SUCCESS or FAILURE :

    frontier = Queue.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
        state = frontier.dequeue()
        explored.add(state)

        if goalTest(state):
            return SUCCESS(state)

        for neighbor in state.neighbors():
            if neighbor not in frontier  $\cup$  explored:
                frontier.enqueue(neighbor)

    return FAILURE
```

Tìm kiếm theo chiều rộng

- Hãy làm rõ quá trình biến đổi của danh sách **frontier** ?

frontier = {A}

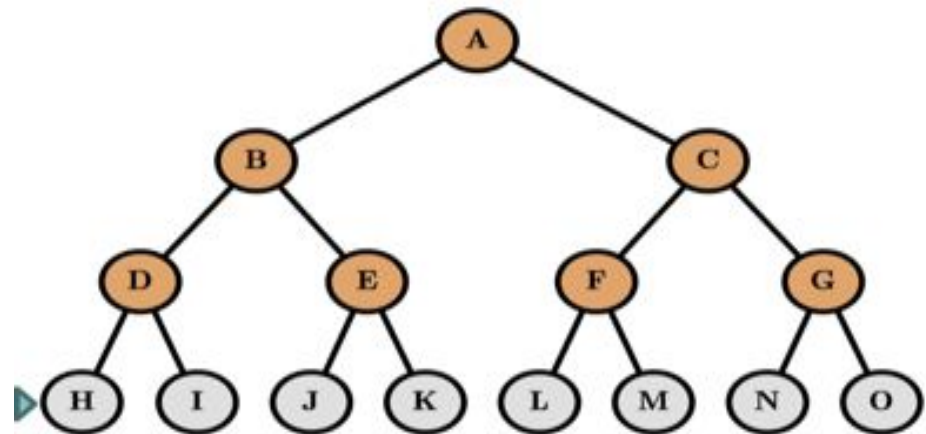
frontier = {B, C}

frontier = {C, D, E}

frontier = {D, E, F, G}

frontier = {E, F, G, H, I}

frontier = ...



```
function BREADTH-FIRST-SEARCH(initialState, goalTest)
    returns SUCCESS or FAILURE :
```

```
    frontier = Queue.new(initialState)
    explored = Set.new()
```

```
    while not frontier.isEmpty():
        state = frontier.dequeue()
        explored.add(state)
```

```
        if goalTest(state):
            return SUCCESS(state)
```

```
        for neighbor in state.neighbors():
            if neighbor not in frontier  $\cup$  explored:
                frontier.enqueue(neighbor)
```

```
    return FAILURE
```

Tìm kiếm theo chiều rộng

- Nhận xét về thuật toán
 - Trong thuật toán tìm kiếm theo chiều rộng, đỉnh nào phát sinh ra trước sẽ được duyệt trước, do đó danh sách frontier phải là FIFO. Trạng thái kết thúc sẽ xác định bởi một điều kiện nào đó.
 - Nếu bài toán có nghiệm(tìm thấy đường đi từ đỉnh đầu tới đỉnh đích), thì thuật toán sẽ tìm được nghiệm.

Tìm kiếm theo chiều rộng

- Đánh giá thuật toán theo chiều rộng
 - **Tính đủ ?**
 - có (nếu b hữu hạn)
 - **Thời gian ?**
 - $1 + 1.b + b.b + \dots + b^{(d-1)}. b = O(b^d)$
 - **Không gian?**
 - $O(b^d)$
 - Lưu ý: trường hợp $O(b^{(d+1)})$.
 - **Tính tối ưu ?**
 - có (nếu chi phí cho mỗi bước chuyển là 1 đơn vị).
- Độ phức tạp về thời gian và không gian của thuật toán tìm kiếm theo chiều rộng theo cấp số nhân. Tại sao sử dụng thuật toán ?

Tìm kiếm theo chiều rộng

- Tìm kiếm theo chiều rộng tệ đến mức nào ?

Đ
BACH KHOA

N
A
N
G

Tìm kiếm theo chiều rộng

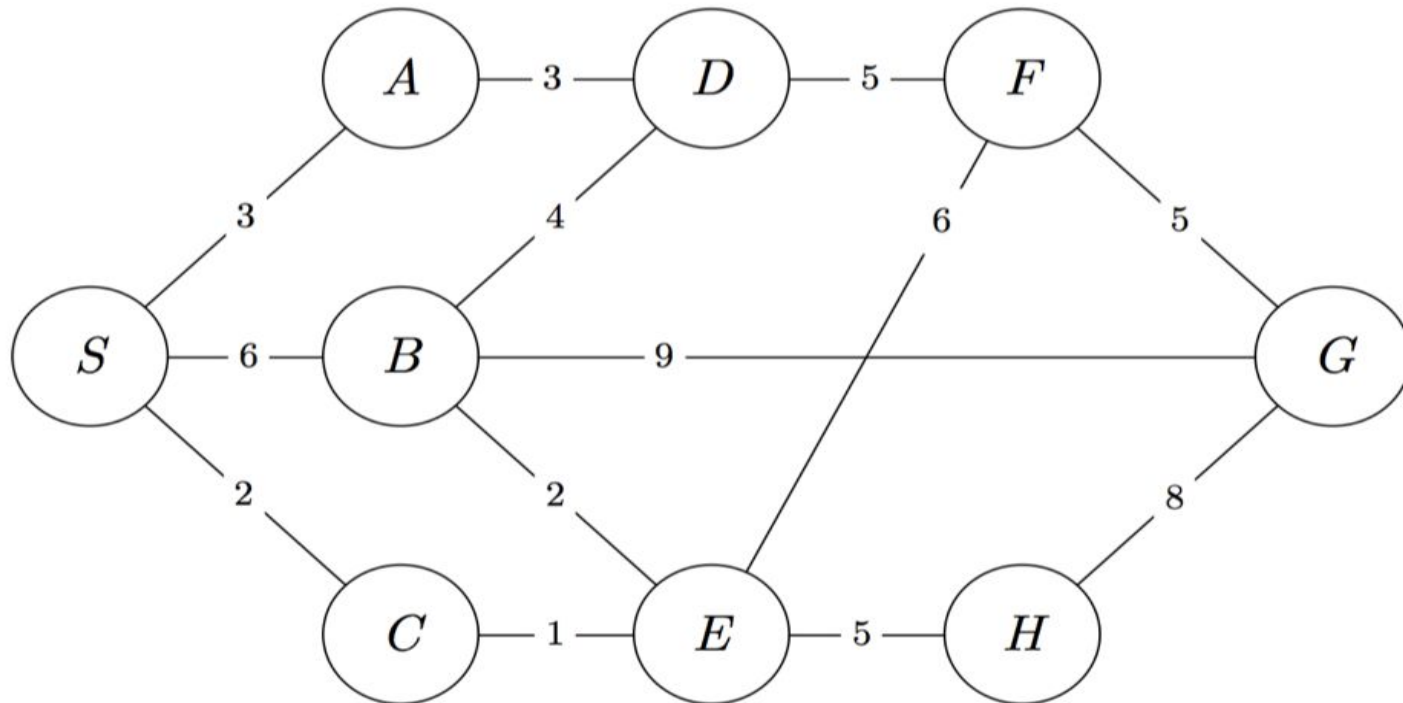
- Tìm kiếm theo chiều rộng tệ đến mức nào ?

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

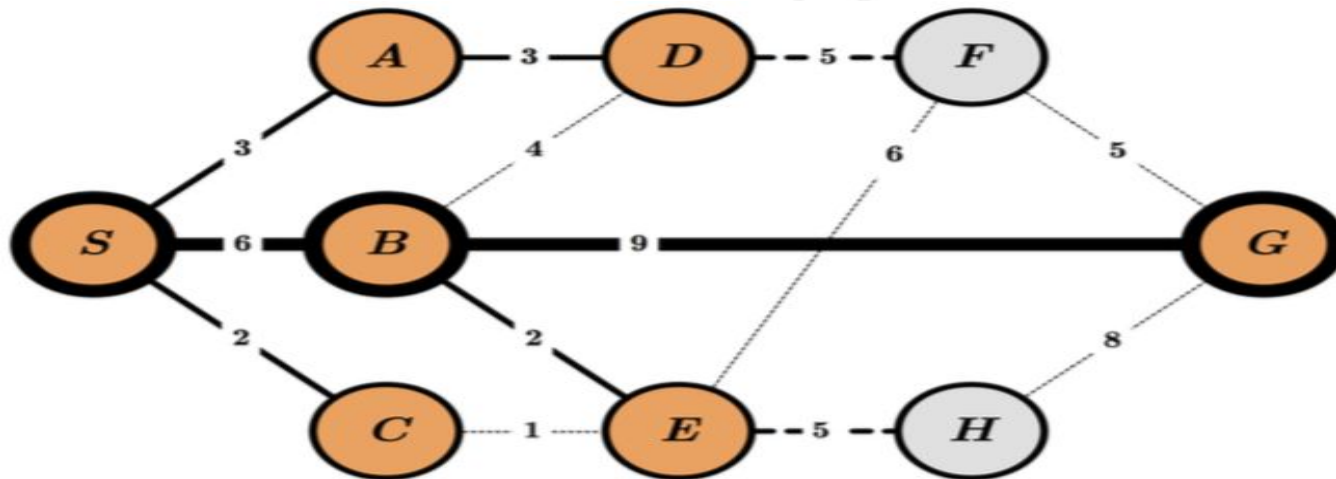
- $b = 10$, 1 giây kiểm tra được 1 triệu nodes, lưu 1 node mất 1.000byte
- **Độ phức tạp thời gian và không gian(bộ nhớ) là bất lợi của thuật toán**

Bài tập

- Hãy xây dựng thứ tự truy cập và đường đi của các đỉnh sử dụng tìm kiếm theo chiều rộng.



Đáp án: BFS



Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>G</i>	<i>F</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B* *C* *D* *E* *G*



ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

Khoa Công Nghệ Thông Tin
TS. Nguyễn Văn Hiệu



Demo

D
BACH KHOA

N
A
N
G

Tìm kiếm đều giá (Uniform-cost search)

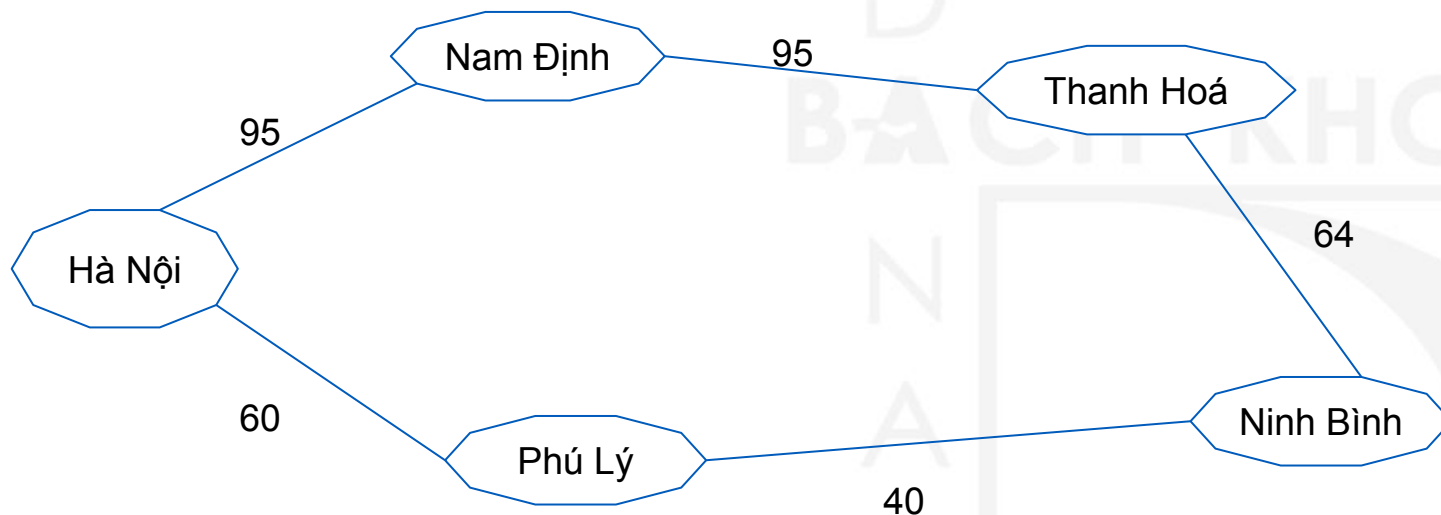


Tìm kiếm đều giá

- Tương như tìm kiếm theo chiều rộng nếu các đỉnh có “giá” như nhau
- Ý tưởng: xét đỉnh có “giá” nhỏ nhất trước
- Cài đặt: sử dụng hàng đợi có sự ưu tiên về “giá”
- Cải tiến BFS: ưu tiên chi phí, không ưu tiên độ sâu.
- Mỗi đỉnh n viếng thăm có hàm chi phí bé nhất $g(n)$

Tìm kiếm đều giá

- Mỗi đỉnh n viếng thăm có hàm chi phí bé nhất $g(n)$



- Từ Hà Nội đến Thanh Hoá. Sử dụng BFS, thì cho đường đi Hà Nội - Nam Định - Thanh Hoá, nhưng sử dụng UCS, thì cho đường đi Hà Nội - Phú Lý - Ninh Bình - Thanh Hóa với khoảng cách ngắn hơn.

Tìm kiếm đều giá

- Mỗi đỉnh n viếng thăm có hàm chi phí bé nhất $g(n)$
- Thuật toán tìm kiếm đều giá được mô tả bởi thủ tục:

```
function UNIFORM-COST-SEARCH(initialState, goalTest)
  returns SUCCESS or FAILURE : /* Cost  $f(n) = g(n)$  */

  frontier = Heap.new(initialState)
  explored = Set.new()

  while not frontier.isEmpty():
    state = frontier.deleteMin()
    explored.add(state)

    if goalTest(state):
      return SUCCESS(state)

    for neighbor in state.neighbors():
      if neighbor not in frontier  $\cup$  explored:
        frontier.insert(neighbor)
      else if neighbor in frontier:
        frontier.decreaseKey(neighbor)

  return FAILURE
```

Cập nhật “giá”

Tìm kiếm đều giá

- Hãy làm rõ quá trình biến đổi của danh sách **frontier** ?

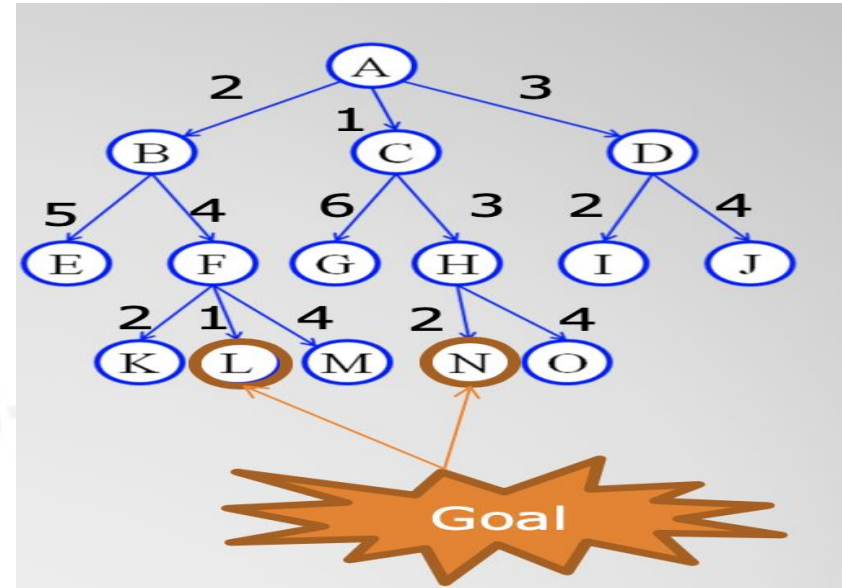
frontier = {A(0)}

frontier = {B(2), C(1), D(3)}

frontier = {B(2), G(1+6), H(1+3), D(3)}

frontier = {E(2+5), F(2+4), G(7), H(4), D(3)}

frontier = ...



function UNIFORM-COST-SEARCH(initialState, goalTest)
returns **SUCCESS** or **FAILURE** : /* Cost $f(n) = g(n)$ */

frontier = Heap.new(initialState)
explored = Set.new()

while not frontier.isEmpty():
state = frontier.deleteMin()
explored.add(state)

if goalTest(state):
return **SUCCESS**(state)

for neighbor **in** state.neighbors():
if neighbor **not in** frontier $\cup$ explored:
frontier.insert(neighbor)
else if neighbor **in** frontier:
frontier.decreaseKey(neighbor)

return **FAILURE**

Tìm kiếm theo chiều sâu

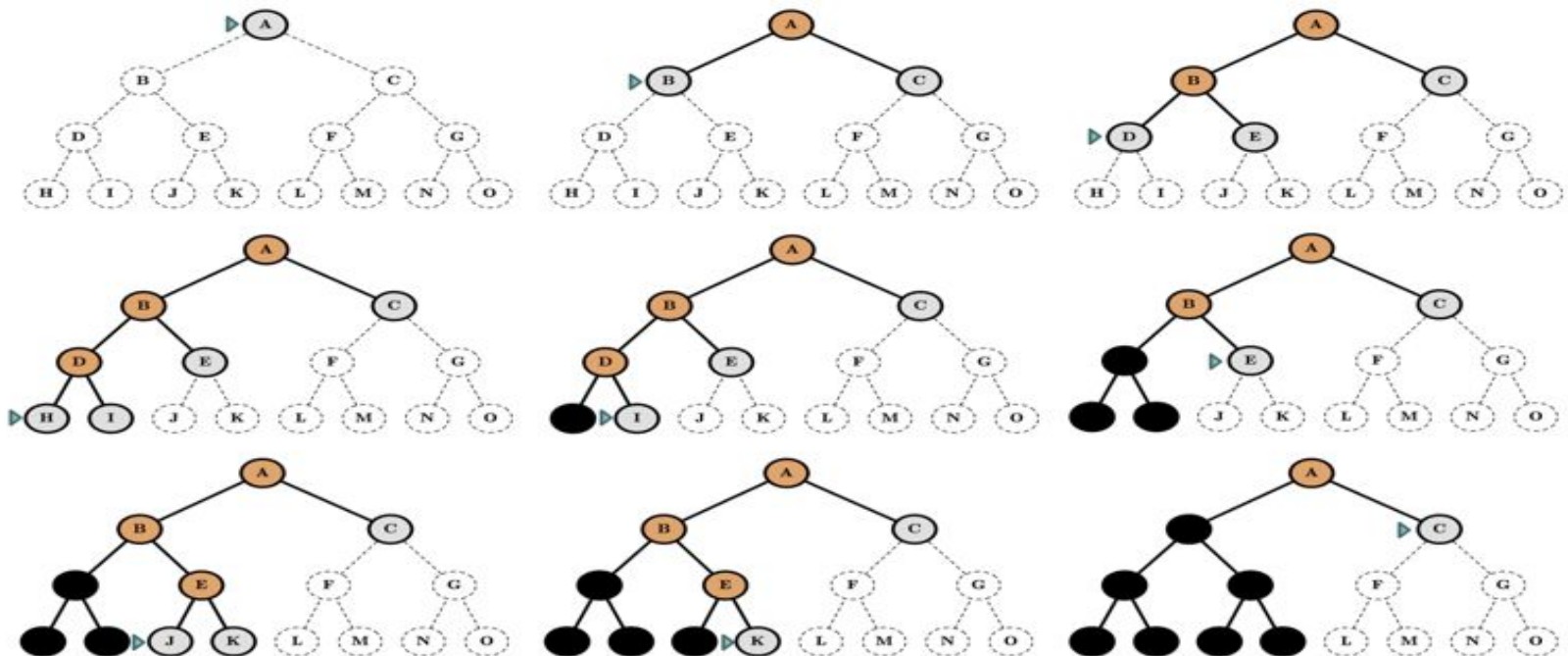


Tìm kiếm theo chiều sâu

- Phát triển các đỉnh ở sâu so với đỉnh hiện tại
- Tập đỉnh được chia thành 3 khu vực
 - Khu vực đã phát triển(explored): tập đỉnh đã xét
 - Khu vực chờ phát triển(frontier): tập đỉnh đang chờ xét
 - Khu vực chưa phát triển(unexplored): tập đỉnh chưa xét

Tìm kiếm theo chiều sâu

- Phát triển các đỉnh ở sâu so với đỉnh hiện tại
- Minh họa việc phát triển các đỉnh theo thuật toán tìm kiếm theo chiều sâu



Tìm kiếm theo chiều sâu

- Phát triển các đỉnh ở sâu so với đỉnh hiện tại
- Thuật toán tìm kiếm theo chiều sâu được mô tả bởi thủ tục

```
function DEPTH-FIRST-SEARCH(initialState, goalTest)
    returns SUCCESS or FAILURE :

    frontier = Stack.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
        state = frontier.pop()
        explored.add(state)

        if goalTest(state):
            return SUCCESS(state)

        for neighbor in state.neighbors():
            if neighbor not in frontier  $\cup$  explored:
                frontier.push(neighbor)

    return FAILURE
```

Tìm kiếm theo chiều sâu

- Hãy làm rõ quá trình biến đổi của danh sách **frontier** ?

Frontier = {A}

Frontier = {C, B,}

Frontier = {C, E, D}

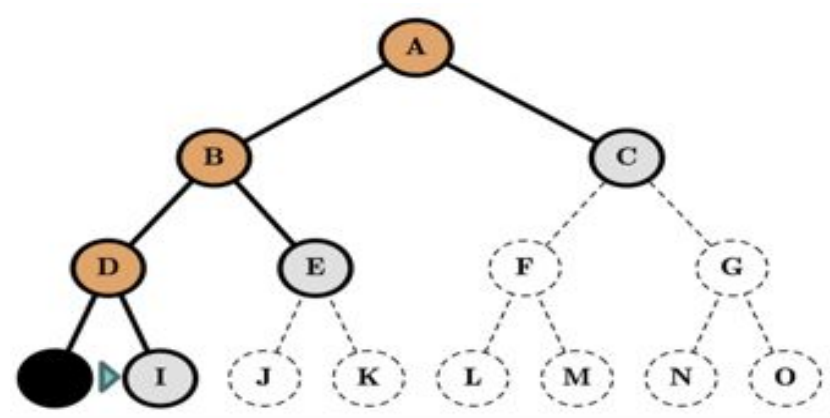
Frontier = {C, E, I, H}

Frontier = {C, E, I}

Frontier = {C, E}

Frontier = {C, J, K}

Frontier = {....}



```
function DEPTH-FIRST-SEARCH(initialState, goalTest)
    returns SUCCESS or FAILURE :

    frontier = Stack.new(initialState)
    explored = Set.new()

    while not frontier.isEmpty():
        state = frontier.pop()
        explored.add(state)

        if goalTest(state):
            return SUCCESS(state)

        for neighbor in state.neighbors():
            if neighbor not in frontier ∪ explored:
                frontier.push(neighbor)

    return FAILURE
```

Tìm kiếm theo chiều sâu

- Nhận xét về thuật toán
 - Đối với BFS luôn tìm được nghiệm (nếu bài toán có nghiệm)
 - Đối với DFS chỉ tìm được nghiệm(nếu không gian hữu hạn), vì nếu không gian tìm kiếm vô hạn, thì thuật toán tìm kiếm theo chiều sâu chọn nhánh để đi, nếu đi đúng nhánh vô hạn mà nghiệm không nằm trên nhánh đó.

Tìm kiếm theo chiều sâu

- Đánh giá thuật toán theo chiều rộng
 - **Tính đủ ?**
 - không (nếu không gian tìm kiếm vô hạn hoặc lặp)
 - đủ (nếu khử lặp và không gian tìm kiếm hữu hạn)
 - **Thời gian ?**
 - $O(b^m)$
 - Nếu hàm đánh giá tốt thì, thời gian thực tế giảm đáng kể
 - **Không gian?**
 - $O(b.m)$: độ phức tạp tuyến tính
 - **Tính tối ưu ?**
 - không

Tìm kiếm theo chiều sâu

- Quay trở lại với bảng đánh giá tìm kiếm theo chiều rộng

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

$b = 10$, 1 giây kiểm tra được 1 triệu nodes, lưu 1 node mất 1.000byte

- Với $d = 16$, có thể giảm dung lượng bộ nhớ từ 10 exabyte trên BFS xuống bao nhiêu trên tìm kiếm theo chiều sâu(DFS) ?

Tìm kiếm theo chiều sâu

- Quay trở lại với bảng đánh giá tìm kiếm theo chiều rộng

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

$b = 10$, 1 giây kiểm tra được 1 triệu nodes, lưu 1 node mất 1.000byte

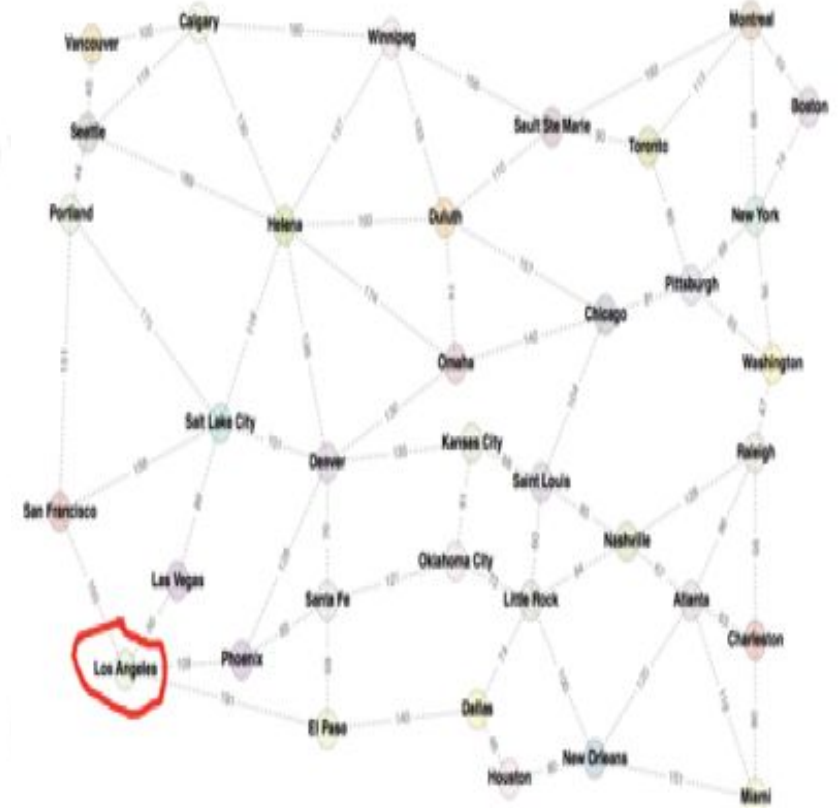
- Với $d = 16$, có thể giảm dung lượng bộ nhớ từ 10 exabytes trên BFS xuống **156 kilobytes** DFS

Tìm kiếm chiều sâu có giới hạn



Tìm kiếm theo chiều sâu có giới hạn

- Giả thiết nếu biết trước bài toán thì không duyệt hết độ sâu trong thuật toán tìm kiếm theo chiều sâu.
- Chọn độ sâu **L**, để phát triển thuật toán tìm kiếm theo chiều sâu (các đỉnh ở độ sâu L, không chứa đỉnh con, không chứa đỉnh cháu)
- Ý tưởng: Qua thực nghiệm chỉ ra rằng không có một thành phố nào tới thành phố khác với độ sâu vượt quá 36, nên $L = 36$



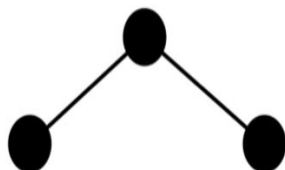
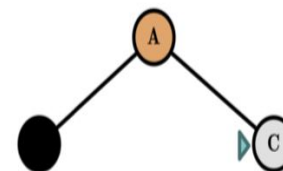
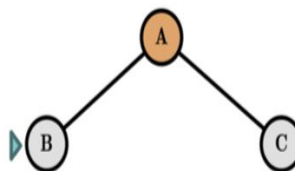
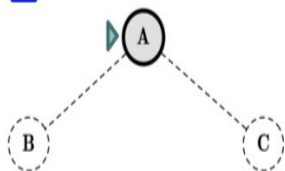
Tìm kiếm theo chiều sâu có giới hạn

Limit = 0



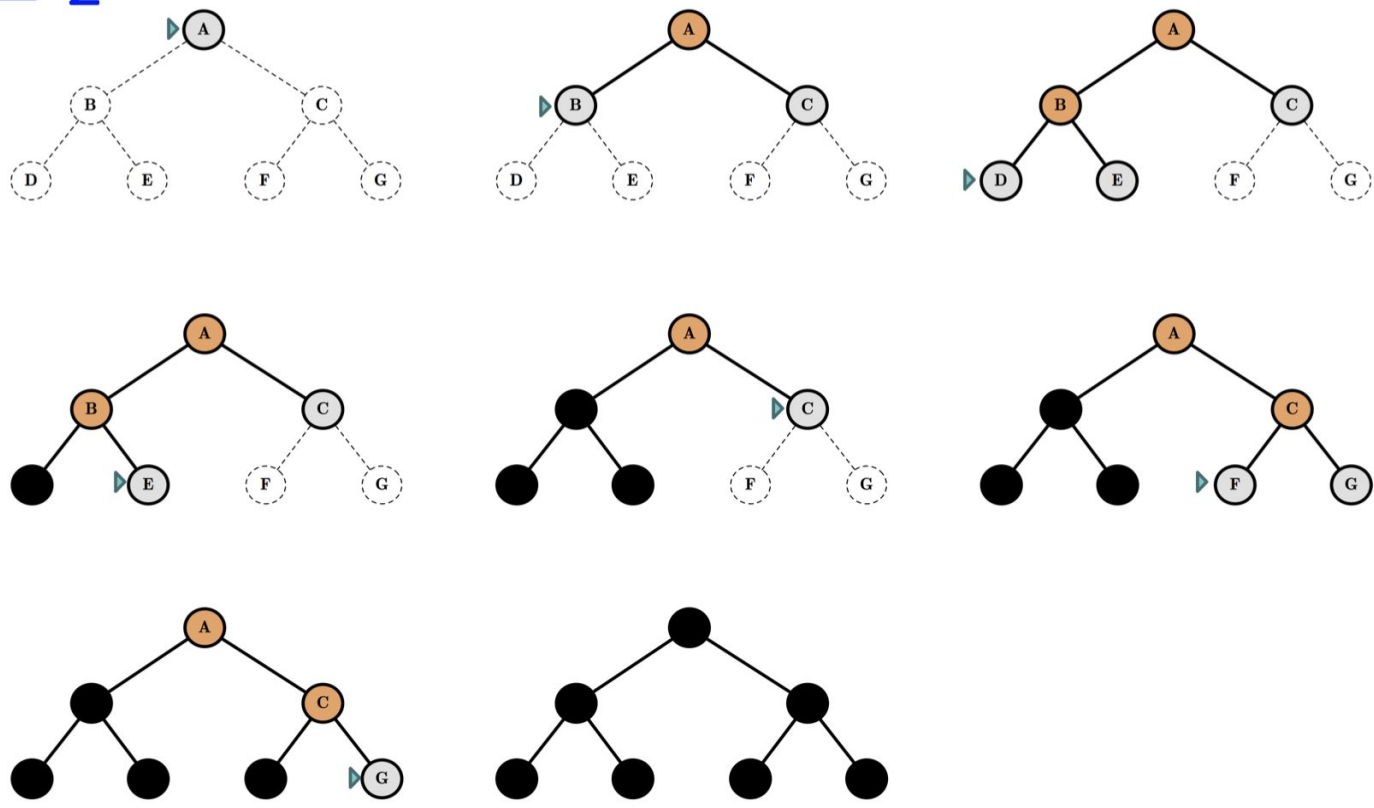
Tìm kiếm theo chiều sâu có giới hạn

Limit = 1



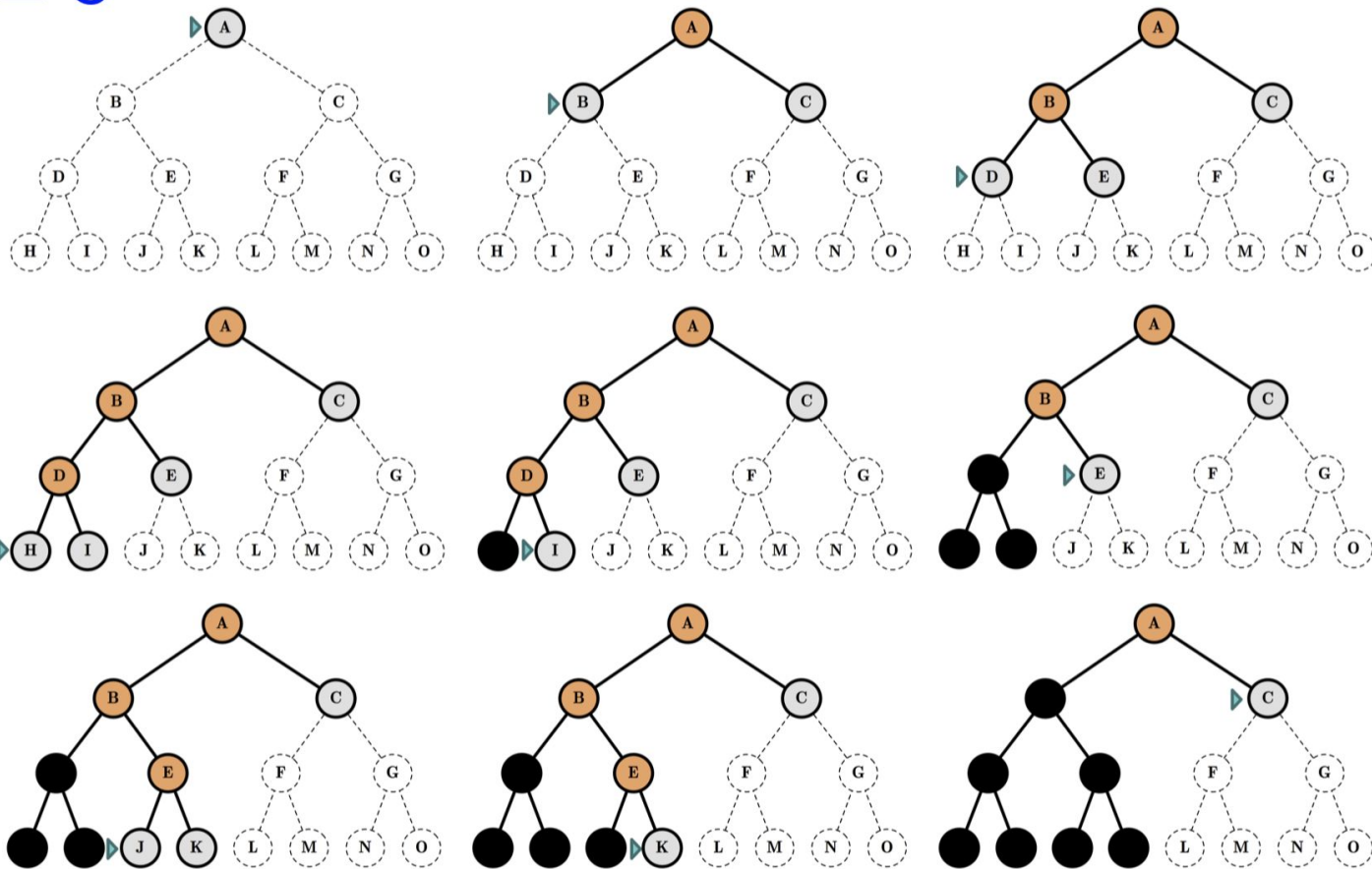
Tìm kiếm theo chiều sâu có giới hạn

Limit = 2



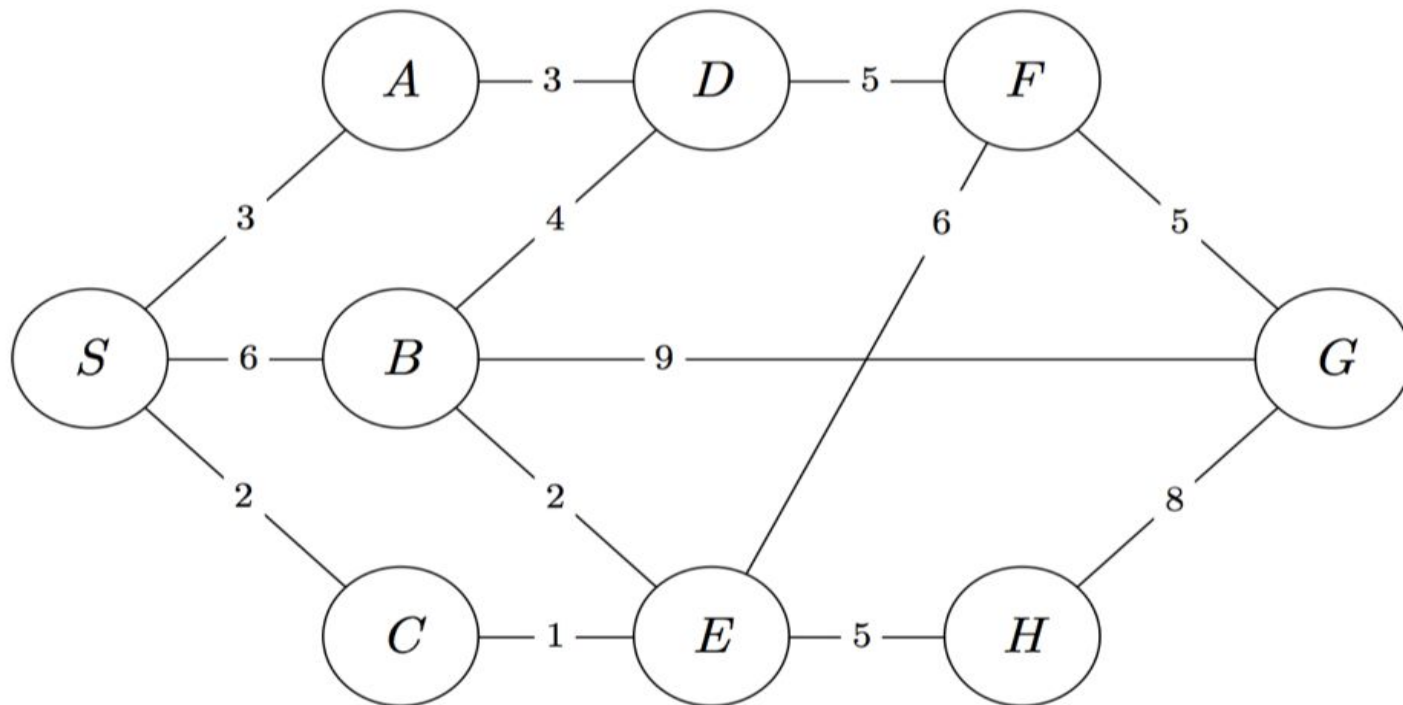
Tìm kiếm theo chiều sâu có giới hạn

Limit = 3



Bài tập 1

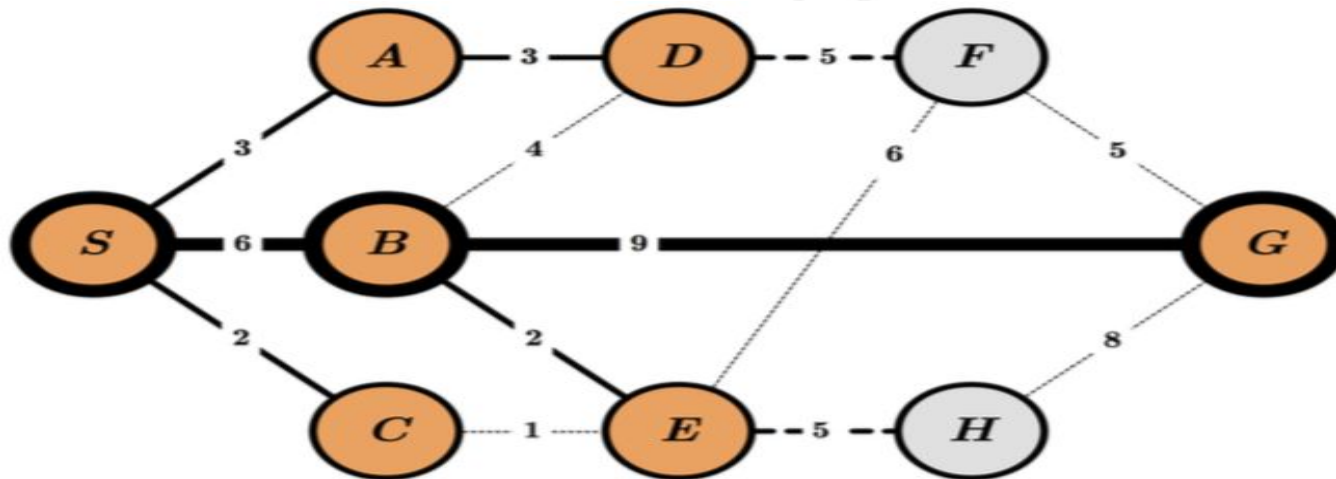
- Hãy xây dựng thứ tự truy cập và đường đi của các đỉnh sử dụng tìm kiếm theo chiều rộng, tìm kiếm theo chiều sâu và tìm kiếm đều giá.



Bài tập 2

- Hãy viết chương trình với các thuật toán tìm kiếm theo chiều rộng, thuật toán tìm kiếm theo chiều sâu và thuật toán tìm kiếm đều giá.

Đáp án bài tập 1: BFS



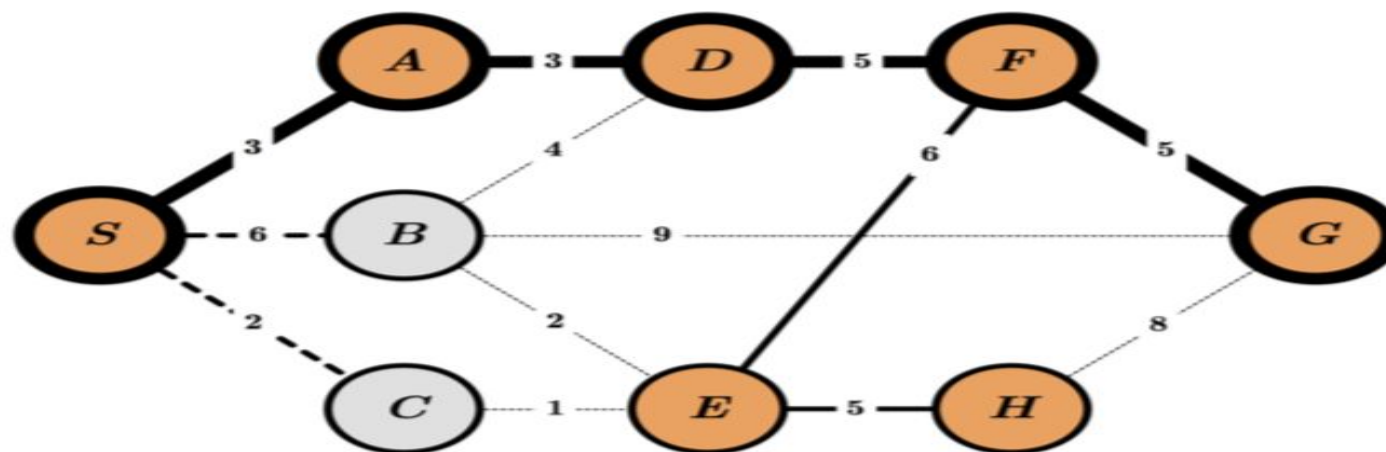
Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>G</i>	<i>F</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B* *C* *D* *E* *G*

Đáp án bài tập 1: DFS



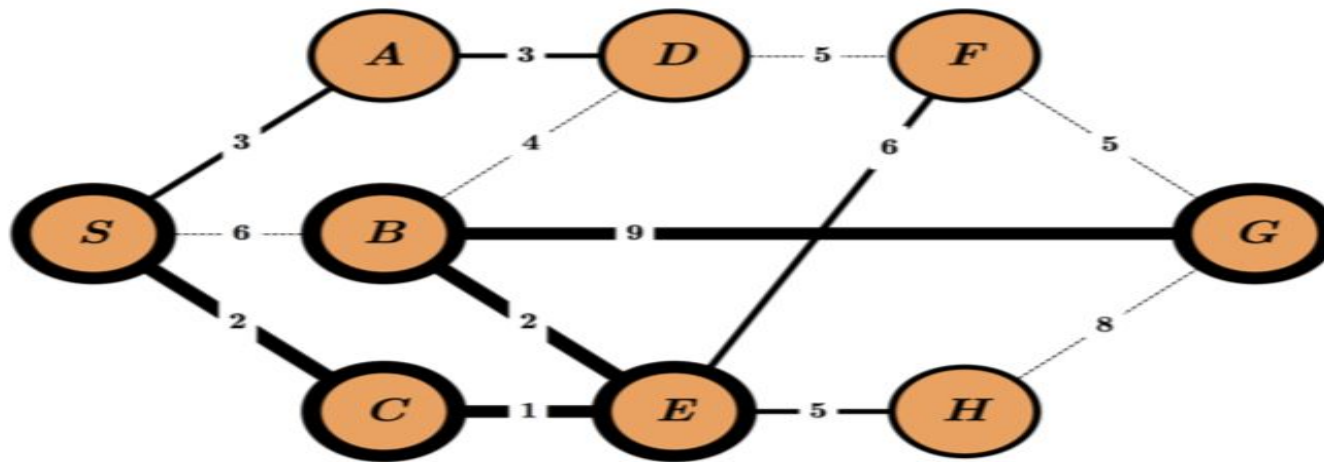
Stack:

<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E* *H* *G*

Đáp án bài tập 1: UCS



Priority Queue:

S_0	C_2	A_3	E_3	B_5	D_6	H_8	F_9	G_{14}
-------	-------	-------	-------	-------	-------	-------	-------	----------

Order of Visit:

S C A E B D H F G